

SQL SERVER 2025

TABLE PARTITIONING

End-to-End Technical Training SOP

**Design • RANGE RIGHT • Partition Functions • Partition Schemes
Aligned Indexes • Partition Elimination • Incremental Statistics
Sliding Window • SPLIT • SWITCH • MERGE • Monitoring • Troubleshooting**

Training Scenario

AdventureWorks2025 | ~50 GB | ~10 Million Rows | Monthly OrderDate Partitions | RANGE RIGHT

Prepared for DBA / Database Engineer / Production Support Training

Microsoft-aligned independent training material

Contents

Section	Topic
1	Purpose, Scope and Learning Objectives
2	Partitioning Fundamentals and Architecture
3	Training Scenario and Design Decisions
4	RANGE RIGHT and Boundary Mathematics
5	Filegroups, Data Files and Storage Strategy
6	Partition Function and Partition Scheme
7	Creating the Partitioned Table
8	Row Routing and Partition Verification
9	Partition Elimination and Query Design
10	Aligned and Nonaligned Indexes
11	Incremental Statistics
12	Sliding-Window Lifecycle
13	SPLIT, NEXT USED, SWITCH and MERGE
14	Partition-Level Index Maintenance
15	Filegroup Backup / Restore Considerations
16	Monitoring and Metadata Queries
17	Production Troubleshooting
18	Common Mistakes and Best Practices
19	End-to-End Production Runbook
20	DBA Validation Checklist and Training Takeaways
21	Microsoft Reference Documentation

1. Purpose, Scope and Learning Objectives

This SOP teaches SQL Server table and index partitioning from design through day-2 operations. The goal is to understand not only how to create partitions, but how to operate a large partitioned table safely in production.

- Understand partition columns, functions, schemes, partitions and filegroups.
- Design monthly date-based partitions using RANGE RIGHT.
- Create and verify a partitioned table and partition-aligned indexes.
- Understand partition elimination and why partitioning is not an automatic performance fix.
- Implement a sliding-window strategy using SPLIT, SWITCH and MERGE.
- Use incremental statistics and partition-level index maintenance appropriately.
- Monitor row distribution, boundaries, storage and dependencies using SQL Server metadata.
- Apply production safeguards before changing shared partition functions or switching data.

Core Principle

SQL Server partitioning is primarily a manageability and scalability feature. Performance can improve when the optimizer eliminates irrelevant partitions, but good indexing, statistics and query design are still required.

2. Partitioning Fundamentals and Architecture

Partitioning horizontally divides one logical table or index into multiple physical partitions. Applications continue to query one table name; SQL Server routes rows and accesses partitions internally.

```

APPLICATION
|
v
dbo.SalesHistory
|
v
Partition Column: OrderDate
|
v
Partition Function: pf_SalesHistory_Monthly
|
v
Partition Scheme: ps_SalesHistory_Monthly
|
+--> Partition 1 --> Filegroup / storage
+--> Partition 2 --> Filegroup / storage
+--> Partition 3 --> Filegroup / storage
+--> ...
|
v
Aligned indexes + statistics + maintenance
  
```

Component	Purpose
Partition column	Column whose value determines row placement.
Partition function	Defines boundary values and maps values to partition numbers.
Partition scheme	Maps function partitions to filegroups.
Partitioned table/index	Stores rows/index entries according to the scheme.
Aligned index	Uses compatible partitioning so table/index partitions correspond.
\$PARTITION	Returns the partition number for a supplied partition-key value.

3. Training Scenario and Design Decisions

Item	Training Design
Database	AdventureWorks2025
Table	dbo.SalesHistory
Approximate size	50 GB
Approximate rows	10 million
Partition key	OrderDate
Granularity	Monthly
Boundary strategy	RANGE RIGHT
Retention example	24 months
Operational model	Sliding window with empty edge capacity

A monthly date key is a common candidate when ingestion, retention, reporting and archival naturally follow calendar periods. The partition key should be chosen from the workload and lifecycle requirements, not simply because it is a date column.

4. RANGE RIGHT and Boundary Mathematics

With RANGE RIGHT, the boundary value belongs to the partition on its right. For monthly partitions, month-start boundaries are therefore intuitive.

Boundary values:

2026-02-01

2026-03-01

2026-04-01

RANGE RIGHT produces:

Partition 1 : OrderDate < 2026-02-01

Partition 2 : OrderDate >= 2026-02-01 AND OrderDate < 2026-03-01

Partition 3 : OrderDate >= 2026-03-01 AND OrderDate < 2026-04-01

Partition 4 : OrderDate >= 2026-04-01

Boundary Rule

Number of partitions = number of boundary values + 1. Always document the intended lower and upper ranges before creating the function.

5. Filegroups, Data Files and Storage Strategy

Partitioning does not require one filegroup per partition. A single-filegroup design can still provide partitioning benefits. Use multiple filegroups when operational requirements justify them, such as storage placement or file/filegroup backup and restore.

```
ALTER DATABASE AdventureWorks2025 ADD FILEGROUP FG_202601;
ALTER DATABASE AdventureWorks2025 ADD FILEGROUP FG_202602;
ALTER DATABASE AdventureWorks2025 ADD FILEGROUP FG_202603;
ALTER DATABASE AdventureWorks2025 ADD FILEGROUP FG_FUTURE;
GO
```

Production Design Warning

Do not create dozens or hundreds of filegroups automatically just because you have monthly partitions. Filegroup architecture should be driven by recovery, storage and operational requirements.

6. Partition Function and Partition Scheme

Step-1 - Create the Partition Function

```
CREATE PARTITION FUNCTION pf_SalesHistory_Monthly (date)
AS RANGE RIGHT
FOR VALUES
(
    '2026-02-01',
    '2026-03-01',
    '2026-04-01'
);
GO
```

Step-2 - Create the Partition Scheme

```
CREATE PARTITION SCHEME ps_SalesHistory_Monthly
AS PARTITION pf_SalesHistory_Monthly
TO
(
    FG_202601,
    FG_202602,
    FG_202603,
    FG_FUTURE
);
GO
```

The function answers “which partition?” The scheme answers “which filegroup?”

7. Creating the Partitioned Table

Step-3 - Create the Table on the Scheme

```
CREATE TABLE dbo.SalesHistory
(
    SalesID      bigint      NOT NULL,
    OrderDate    date        NOT NULL,
    CustomerID   int         NOT NULL,
    ProductID    int         NOT NULL,
    Quantity     int         NOT NULL,
    SalesAmount  decimal(18,2) NOT NULL
)
ON ps_SalesHistory_Monthly(OrderDate);
GO
```

The ON clause places the heap/table data on the partition scheme using OrderDate as the partitioning column. Key and index design must be reviewed carefully for uniqueness and alignment requirements in a real production schema.

8. Row Routing and Partition Verification

Step-4 - Insert a Test Row

```
INSERT dbo.SalesHistory
(
    SalesID, OrderDate, CustomerID, ProductID, Quantity, SalesAmount
)
VALUES
(
    10001, '2026-03-15', 5001, 100, 2, 299.99
);
GO
```

Step-5 - Verify the Row's Partition

```
SELECT
    SalesID,
    OrderDate,
    $PARTITION.pf_SalesHistory_Monthly(OrderDate) AS PartitionNumber
FROM dbo.SalesHistory;
GO
```

Step-6 - Check Row Counts per Partition

```
SELECT
    p.partition_number,
    p.rows
FROM sys.partitions AS p
WHERE p.object_id = OBJECT_ID(N'dbo.SalesHistory')
    AND p.index_id IN (0,1)
ORDER BY p.partition_number;
GO
```

9. Partition Elimination and Query Design

Partition elimination occurs when the optimizer can determine that partitions cannot contain qualifying rows and avoids accessing them. The actual execution plan is the evidence; do not assume elimination simply because a predicate references the partition key.

```
SELECT SUM(SalesAmount)
FROM dbo.SalesHistory
WHERE OrderDate >= '2026-03-01'
    AND OrderDate < '2026-04-01';
GO
```

Query pattern	Training guidance
Direct date range	Preferred: clear lower/upper range on the partition key.
Function applied to key	Be cautious; verify whether the optimizer can still eliminate partitions.
No partition-key restriction	Potentially accesses many/all partitions.
Parameterized range	Validate actual plans for representative parameter values.

Performance Rule

Partitioning does not replace indexes. A partitioned table can still perform poorly if access paths, statistics or predicates are inappropriate.

10. Aligned and Nonaligned Indexes

Partition alignment is important for partition-level management. An aligned index follows a compatible partitioning strategy so its partitions correspond to the table partitions.

```
CREATE INDEX IX_SalesHistory_OrderDate
ON dbo.SalesHistory
(
    OrderDate,
    CustomerID
)
ON ps_SalesHistory_Monthly(OrderDate);
GO
```

- Prefer aligned indexes when sliding-window SWITCH operations are part of the design.
- Review unique index requirements carefully because the partitioning column affects uniqueness rules.
- A nonaligned index can prevent or complicate partition switching and partition-level maintenance.
- Validate every index before implementing a production SWITCH runbook.

11. Incremental Statistics

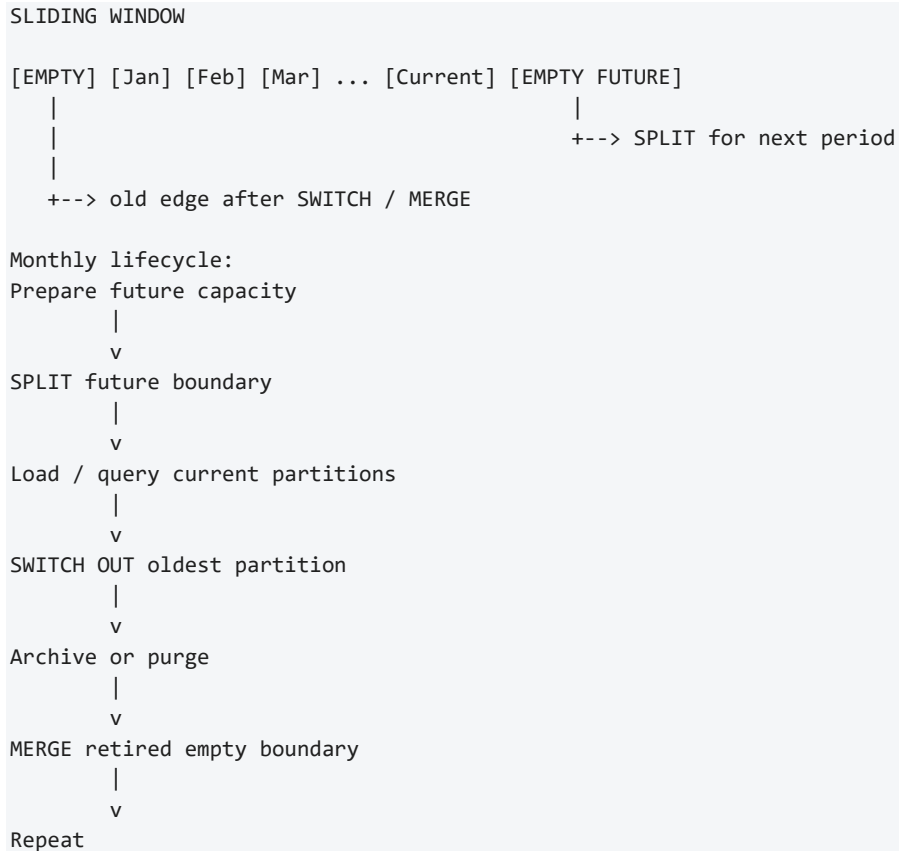
Incremental statistics maintain statistics information at the partition level for supported partitioned indexes/statistics. This is useful when historical partitions are stable while recent partitions change frequently.

```
CREATE INDEX IX_SalesHistory_OrderDate_IncStats
ON dbo.SalesHistory(OrderDate)
WITH
(
    STATISTICS_INCREMENTAL = ON
)
ON ps_SalesHistory_Monthly(OrderDate);
GO
```

Important

Incremental statistics have documented restrictions. Confirm index alignment and feature support for the exact statistics/index design before enabling them.

12. Sliding-Window Lifecycle



The empty-edge strategy minimizes expensive data movement. Splitting or merging populated partitions can be resource intensive, so boundary maintenance should be planned before the new period arrives.

13. SPLIT, NEXT USED, SWITCH and MERGE

Step-7 - Mark the NEXT USED Filegroup

```

ALTER PARTITION SCHEME ps_SalesHistory_Monthly
NEXT USED FG_FUTURE;
GO
  
```

Step-8 - SPLIT the Future Boundary

```

ALTER PARTITION FUNCTION pf_SalesHistory_Monthly()
SPLIT RANGE ('2026-05-01');
GO
  
```

Step-9 - SWITCH OUT the Oldest Partition

```

-- The destination table must satisfy SQL Server SWITCH requirements.
ALTER TABLE dbo.SalesHistory
SWITCH PARTITION <partition_number>
TO dbo.SalesHistory_Archive;
GO
  
```

SWITCH is powerful because it changes allocation ownership rather than performing a conventional row-by-row delete/insert operation. However, SQL Server enforces strict structural, constraint, index and partition compatibility rules.

Step-10 - MERGE the Retired Empty Boundary

```
ALTER PARTITION FUNCTION pf_SalesHistory_Monthly()
MERGE RANGE ('2026-02-01');
GO
```

Safety Gate Before SWITCH

Validate source/destination structure, partition boundary/check constraints, index definitions and alignment, compression and other applicable SWITCH requirements before production execution.

14. Partition-Level Index Maintenance

```
ALTER INDEX IX_SalesHistory_OrderDate
ON dbo.SalesHistory
REBUILD PARTITION = 12;
GO
```

Partition-level maintenance can reduce the scope of work on very large tables. It should still be driven by measured need rather than a blanket rebuild schedule.

- Measure fragmentation/page density and workload impact.
- Focus on actively changing partitions where appropriate.
- Coordinate maintenance with statistics strategy.
- Account for edition/version capabilities and online/resumable options separately.

15. Filegroup Backup / Restore Considerations

A multi-filegroup partition design can support advanced backup/restore and archival strategies. This is an operational reason to distribute partitions, but it is not required for partition elimination.

```
Historical filegroups ---> relatively static data
Current filegroups  ---> active read/write data
Future filegroup    ---> prepared capacity
```

```
Partitioning
|
+--> lifecycle management
+--> storage placement
+--> file/filegroup recovery strategy
```

16. Monitoring and Metadata Queries

Step-11 - List Partition Functions and Boundaries

```
SELECT
    pf.name AS PartitionFunction,
    pf.type_desc,
    pf.boundary_value_on_right,
    prv.boundary_id,
    CONVERT(varchar(30), prv.value, 121) AS BoundaryValue
FROM sys.partition_functions AS pf
LEFT JOIN sys.partition_range_values AS prv
    ON pf.function_id = prv.function_id
WHERE pf.name = N'pf_SalesHistory_Monthly'
ORDER BY prv.boundary_id;
GO
```

Step-12 - Inspect Partition Scheme Mapping

```
SELECT
    ps.name AS PartitionScheme,
    dds.destination_id,
    fg.name AS FilegroupName
FROM sys.partition_schemes AS ps
JOIN sys.destination_data_spaces AS dds
    ON ps.data_space_id = dds.partition_scheme_id
JOIN sys.filegroups AS fg
    ON dds.data_space_id = fg.data_space_id
WHERE ps.name = N'ps_SalesHistory_Monthly'
ORDER BY dds.destination_id;
GO
```

Step-13 - Partition Row and Space View

```
SELECT
    p.partition_number,
    p.rows,
    au.type_desc,
    SUM(au.total_pages) * 8.0 / 1024 AS TotalMB,
    SUM(au.used_pages) * 8.0 / 1024 AS UsedMB
FROM sys.partitions AS p
JOIN sys.allocation_units AS au
    ON p.hobt_id = au.container_id
WHERE p.object_id = OBJECT_ID(N'dbo.SalesHistory')
GROUP BY p.partition_number, p.rows, au.type_desc
ORDER BY p.partition_number;
GO
```

Dependency Warning

A partition function can be shared by multiple partition schemes, tables and indexes. Identify dependencies before SPLIT or MERGE because ALTER PARTITION FUNCTION can affect every object that uses it.

17. Production Troubleshooting

Symptom	What to Check	Likely Direction
Query scans many partitions	Actual plan, predicate, partition key, parameter values	Confirm partition elimination and indexing.
SPLIT is slow	Is target partition populated?	Prefer splitting empty edge partitions.
MERGE is expensive	Rows on either side of boundary	Empty/switch data before merge when possible.
SWITCH fails	Constraints, indexes, schema, alignment, target state	Correct compatibility mismatch.
Rows in unexpected partition	Boundary definitions and RANGE LEFT/RIGHT	Validate with \$PARTITION.
Future inserts fail operational expectations	Future boundary/filegroup preparation	Run monthly boundary-preparation job/runbook.
Index maintenance too large	Index alignment and target partition	Use partition-level maintenance where justified.
Statistics stale on active period	Incremental stats/support and modification patterns	Update appropriate statistics/partitions.

18. Common Mistakes and Best Practices

Avoid	Preferred Practice
Partitioning small tables for fashion	Partition only when manageability/lifecycle/workload justifies it.
Assuming automatic performance gains	Validate partition elimination and actual execution plans.
Poor partition key	Choose a key aligned to data lifecycle and common access patterns.
Too many tiny partitions	Choose practical granularity based on data volume and operations.
No future boundary	Prepare future empty capacity before the period begins.
SPLIT/MERGE on populated ranges at peak time	Operate on empty edges where possible.
Nonaligned indexes without reason	Keep operationally important indexes aligned.
Ignoring statistics	Plan statistics maintenance; evaluate incremental statistics.
One filegroup per partition by default	Use filegroups only when operationally justified.
Changing shared function blindly	Map all dependencies first.
DELETE for large retention purge by default	Evaluate SWITCH-based sliding window.
SWITCH without pre-validation	Use a strict compatibility checklist and tested runbook.

19. End-to-End Production Runbook

```

PRE-CHECK
|
+--> Confirm backup/recovery posture
+--> Confirm partition function dependencies
+--> Validate current boundaries
+--> Validate row counts and empty edge partitions
+--> Validate aligned indexes
+--> Validate future filegroup / NEXT USED where applicable
|
v
PREPARE NEXT PERIOD
|
+--> NEXT USED
+--> SPLIT future empty boundary
+--> Verify new partition mapping
|
v
NORMAL OPERATIONS
|
+--> Monitor rows / size
+--> Validate elimination for critical queries
+--> Maintain active indexes/statistics
|
v
RETENTION CYCLE
|
+--> Prepare compatible archive/staging table
+--> Validate SWITCH prerequisites
+--> SWITCH OUT oldest partition
+--> Archive / purge switched table
+--> MERGE retired empty boundary
|
v
POST-CHECK
|
+--> Verify partition count and boundaries
+--> Verify row counts
+--> Verify filegroup mapping
+--> Validate critical queries
+--> Record operational evidence

```

20. DBA Validation Checklist and Training Takeaways

- Partition key matches the business/data lifecycle.
- RANGE RIGHT/LEFT choice is documented and tested with boundary dates.
- Function boundaries and expected partition count are verified.
- Scheme-to-filegroup mapping is documented.
- Critical indexes are intentionally aligned or nonaligned.
- Representative queries are tested for partition elimination.
- Statistics strategy is documented, including incremental statistics where appropriate.
- Empty edge partitions are maintained for the sliding window.
- SPLIT/MERGE are not performed blindly on populated partitions.
- SWITCH prerequisites are validated before execution.

- Partition function dependencies are checked before alteration.
- Row counts, space and boundaries are monitored after every lifecycle operation.
- Backup/restore implications are understood for multi-filegroup designs.

Training Takeaway

A mature partitioning implementation combines data placement, query access, index alignment, statistics, retention, maintenance and recovery into one operational design. CREATE PARTITION FUNCTION is only the beginning.

21. Microsoft Reference Documentation

- Microsoft Learn - Create partitioned tables and indexes
- Microsoft Learn - Partitioned tables and indexes
- Microsoft Learn - ALTER PARTITION FUNCTION (Transact-SQL)
- Microsoft Learn - Modify a partition function
- Microsoft Learn - ALTER TABLE (Transact-SQL) - SWITCH
- Microsoft Learn - Statistics - incremental statistics

Copyright / Attribution Notice

This is independent training material written for educational purposes. It is not official Microsoft courseware and does not reproduce Microsoft documentation as chapters. Microsoft product names and T-SQL interface names are referenced for technical instruction. Microsoft documentation should remain the authoritative source for exact version-specific behavior and restrictions.